

Principles of Functional Programming

Personal Notes

Mihir Rao

Contents

1 Prologue	2
1.1 State, purity, and evaluation	2
1.2 Expressions and values	3
1.3 Types predict permitted behavior	3
1.4 Declarations and function application	4

1 Prologue

A program should both instruct a computer and explain its ideas to the people who read it. Functional programming helps by making pieces easier to reason about separately. Standard ML (SML) supports this with types checked before execution and fewer hidden changes to shared data.

1.1 State, purity, and evaluation

An imperative computation often changes *state*, the program's stored information at a particular moment. Its result can therefore depend on earlier events; this Python function returns a different value after every call.

```
1 count = 0
2 def increment():
3     global count
4     count += 1
5     return count
```

Understanding `increment()` requires the current value of `count`; one part of the program can change what another part observes.

Definition 1.1 [*Pure function*]. The slides say a function is pure if it “does not have any observable side effects” and “always returns the same outputs, given the same inputs” (p. 25). Calling it does not change what other code observes.

If a problem does not need changing state, pure functions keep the solution simpler. We can follow how each expression becomes a value without tracking changes elsewhere in the program.

Note. Purity does not forbid interaction with the outside world. It asks that state be introduced deliberately and isolated when a computation does not require it.

Three principles guide this approach:

1. **Recursive problems, recursive solutions.** Lists, trees, and other recursively defined structures naturally suggest recursive programs.
2. **Programmatic thinking is mathematical thinking.** Evaluation supports algebraic reasoning, correctness proofs, and complexity analysis.
3. **Types guide structure.** Types document interfaces and rule out behaviors that do not fit those interfaces.

1.2 Expressions and values

In SML, computation is evaluation. The basic object being evaluated is an expression.

Definition 1.2 [*Expression*]. The slides call an expression “the building block of an SML program” (p. 32). It may evaluate to a value, raise an exception, or run forever.

Definition 1.3 [*Value*]. A value is “a final answer, that cannot be simplified further” (slide 32). Constants such as 2, “hi”, and true are values; $2 + 2$ is an expression that is not yet a value.

Write $e \Longrightarrow e'$ when expression e takes one evaluation step to e' . Its reflexive, transitive closure is written $\Longrightarrow^*$, meaning zero or more steps. For example,

$$(2 + 3) \cdot 4 \Longrightarrow 5 \cdot 4 \Longrightarrow 20,$$

so $(2 + 3) \cdot 4 \Longrightarrow^* 20$. This sequence is a *computation trace*; the expression evaluates to 20.

Not every well-formed-looking expression yields a value. Evaluating $1 \text{ div } 0$ raises an exception, and a recursive computation with no base case may run forever. Thus evaluation has three possible behaviors:

1. reduction to a value,
2. an exception, or
3. nontermination.

1.3 Types predict permitted behavior

The expression “hi” ^ “ there” concatenates two strings, whereas “1” ^ 50 gives a string operator an integer operand. SML rejects the mismatch instead of inventing a conversion.

Definition 1.4 [*Type*]. A type is “a specification of the behavior of a piece of code” (slide 37). It limits what the code is allowed to do. We write $e : t$ to say that expression e has type t .

The type of a larger expression follows from the types of its parts. For integer addition, the rule is

$$\frac{e_1 : \text{int} \quad e_2 : \text{int}}{e_1 + e_2 : \text{int}}.$$

This proves $1 + (2 + 3) : \text{int}$ by typing each literal, then applying the rule to the inner and outer additions. String concatenation instead requires two string operands; because $50 : \text{int}$, “1” ^ 50 is ill-typed.

Definition: A program *type-checks*, or is *well-typed*, when it obeys the typing rules. A program that violates them is *ill-typed*.

SML is *statically typed*: it checks types before evaluation and does not run ill-typed programs. An expression of type `int` must produce an integer if it produces a value, but it may raise an exception or run forever.

1.4 Declarations and function application

Declarations associate names with typed program components. A function declaration has a name, typed parameters, a declared result type, and a body:

```
1 fun double (n : int) : int = n + n
```

Function application is written by adjacency, so applying `double` to `2` is written `double 2`, and

$$\text{double } 2 \implies 4.$$

The type of `double` is `int -> int`: it accepts an integer and returns an integer. The annotations are checked rather than trusted. Claiming a `string` result while retaining `n + n` would make the body type disagree with the annotation, so SML would reject the declaration.

More generally, function application obeys the rule

$$\frac{e_1 : t_1 \rightarrow t_2 \quad e_2 : t_1}{e_1 \ e_2 : t_2}.$$

The argument type must match the function's input type, and the application has the function's output type.

A value declaration uses `val` and has no parameter list:

```
1 val favoriteCourseNumber : int = 150
```

These ideas let us ask two separate questions about an SML expression: does it obey the typing rules, and what happens when it is evaluated?

Source: Brandon Wu, Carnegie Mellon University 15-150, Principles of Functional Programming, Summer 2023, Lecture 1: "Prologue," official slides and course materials, <https://brandonspark.github.io/prologue/lecture01.pdf> and <https://brandonspark.github.io/150/> (accessed August 13, 2026).